

BENGALURU NORTH UNIVERSITY

SUVARN GANGE,Mangasandra

Kolar -563103

Master of Science in ComputerScience



R-PROGRAMMING LAB

by

Mr. BALAJI B K

Assistant professor

Department of Computer science

FOURTH SEMESTER

2025-26

Lab Manual

CONTENTS

1. Download and install R-Programming environment and install basic packages using `install.packages()` command in R.
2. Implement all the basics of R-Programming (Data types, Variables, Operators etc.,)
3. Implement data frames in R. Write a program to join columns and rows in a data frame using `c bind ()` and `r bind ()` in R.
4. Implement different String Manipulation functions in R.
5. Write R command to
 - i) Illustrate summation, subtraction, multiplication, and division operations on vectors using vectors.
 - ii) Enumerate multiplication and division operations between matrices and vectors in R console.
6. Write R command to
 - i) Illustrates the usage of Vector subsetting and Matrix subsetting
 - ii) Write a program to create an array of 3 X 3 matrixes with 3 rows and 3 columns.
7. Write an R program to draw
 - i) Pie chart ii) 3D Pie Chart iii) Bar Chart along with chart legend by considering suitable CSV file.
8. Create a CSV file having Speed and Distance attributes with 1000 records. Write R program to draw
 - i) Box plots ii) Histogram iii) Line Graph iv) Multiple line graphs v) Scatter plot
9. To demonstrate the relation between the cars speed and the distance.
10. Implement different data structures in R (Vectors, Lists, Data Frames).
11. Write an R program to read a csv file and analyze the data in the file using EDA (Explorative Data Analysis) techniques.
12. Write an R program to illustrate Linear Regression and Multi linear Regression considering suitable CSV file.
13. Create a data set and do statistical analysis on the data using R.
14. Write R program to find Correlation and Covariance
15. Write R program to build classification model using KNN algorithm.
16. Write R program to build clustering model using K-mean algorithm.

1.Download and install R-Programming environment and install basic packages using install. packages() command in R.

• Download R

👉 Go to official website:

🌐 <https://cran.r-project.org>

Steps:

1. Click **Download R for Windows**
2. Click **base**
3. Click **Download R (latest version)**

Install R

After downloading:

1. Open the .exe file
2. Click **Next**
3. Accept license
4. Choose installation location (default is OK)
5. Click **Next** → **Install** → **Finish**

Verify Installation

Open R Console and type:

version

👉 If version appears → installation successful

Install RStudio (Recommended)

👉 RStudio is used for easy coding

Steps:

1. Go to: <https://posit.co/download/rstudio-desktop/>
2. Download **RStudio Desktop**
3. Install like normal software

Open RStudio

👉 After installation:

- Open RStudio
- You will see:
 - Console
 - Script Editor
 - Environment

To install required packages in R programming language.

Program:

```
install.packages("ggplot2")  
install.packages("dplyr")  
library(ggplot2)  
library(dplyr)
```

Expected Output:

Packages are successfully installed and loaded without errors.

2. Implement all the basics of R-Programming (Data types, Variables, Operators etc.,)

Program:

```
# Data Types
num_var <- 42.5
int_var <- as.integer(10)
bool_var <- TRUE
char_var <- "Hello R!"
complex_var <- 3 + 4i
cat("Numeric:", num_var, "\n")
cat("Integer:", int_var, "\n")
cat("Logical:", bool_var, "\n")
cat("Character:", char_var, "\n")
cat("Complex:", complex_var, "\n\n")

# Variables
x <- 25
y <- 10
z <- x + y
cat("x:", x, "\n")
cat("y:", y, "\n")
cat("z (x+y):", z, "\n\n")

# Arithmetic Operators
a <- 10
b <- 5
cat("Addition:", a + b, "\n")
cat("Subtraction:", a - b, "\n")
cat("Multiplication:", a * b, "\n")
cat("Division:", a / b, "\n")
cat("Modulus:", a %% b, "\n")
cat("Exponent:", a ^ b, "\n\n")

# Comparison Operators
cat("Less Than:", a < b, "\n")
cat("Greater Than:", a > b, "\n")
cat("Equal To:", a == b, "\n")
cat("Not Equal:", a != b, "\n")
cat("Less Equal:", a <= b, "\n")
cat("Greater Equal:", a >= b, "\n\n")
```

```
# Logical Operators
p <- TRUE
q <- FALSE
cat("AND:", p & q, "\n")
cat("OR:", p | q, "\n")
cat("NOT:", !p, "\n")
```

Expected Output:

```
Numeric: 42.5
Integer: 10
Logical: TRUE
Character: Hello R!
Complex: 3+4i
x: 25
y: 10
z (x+y): 35
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
Modulus: 0
Exponent: 100000
Less Than: FALSE
Greater Than: TRUE
Equal To: FALSE
Not Equal: TRUE
Less Equal: FALSE
Greater Equal: TRUE
AND: FALSE
OR: TRUE
NOT: FALSE
```

3.Implement data frames in R. Write a program to join columns and rows in a data frame using c bind () and r bind () in R.

Program:

```
# Create Data Frames
df1 <- data.frame(
  Name = c("A", "B"),
  Marks = c(80, 90))
df2 <- data.frame(
  Name = c("C", "D"),
  Marks = c(70, 60))

# Column Bind
df3 <- data.frame(Grade = c("A", "B"))
col_bind <- cbind(df1, df3)

# Row Bind
row_bind <- rbind(df1, df2)
cat("Column Bind:\n")
print(col_bind)
cat("\nRow Bind:\n")
print(row_bind)
```

Expected Output:

```
Column Bind:
Name Marks Grade
1 A 80 A
2 B 90 B
```

```
Row Bind:
Name Marks
1 A 80
2 B 90
3 C 70
4 D 60
```

4.Implement different String Manipulation functions in R.

Program:

```
str1 <- "Hello"
str2 <- "R Programming"

cat("Combined:", paste(str1, str2), "\n")
cat("Combined (no space):", paste0(str1, str2), "\n")
cat("Length:", nchar(str1), "\n")
cat("Uppercase:", toupper(str1), "\n")
cat("Lowercase:", tolower(str2), "\n")
cat("Substring:", substr(str1, 1, 4), "\n")
cat("Pattern Match:", grepl("R", str2), "\n")
cat("Replace First:", sub("R", "Python", str2), "\n")
cat("Replace All:", gsub("a", "X", str2), "\n")

split_str <- strsplit(str2, " ")
print(split_str)
```

Expected Output:

```
Combined: Hello R Programming
Combined (no space): HelloR Programming
Length: 5
Uppercase: HELLO
Lowercase: r programming
Substring: Hell
Pattern Match: TRUE
Replace First: Python Programming
Replace All: R ProgrXmming
[[1]]
[1] "R" "Programming"
```


5. Write R command to

i) Illustrate summation, subtraction, multiplication, and division operations on vectors using vectors.

ii) Enumerate multiplication and division operations between matrices and vectors in R console.

Program:**i)**

```
# Vector Operations
v1 <- c(10, 20, 30)
v2 <- c(2, 4, 5)
cat("Vector 1:", v1, "\n")
cat("Vector 2:", v2, "\n\n")
cat("Addition:", v1 + v2, "\n")
cat("Subtraction:", v1 - v2, "\n")
cat("Multiplication:", v1 * v2, "\n")
cat("Division:", v1 / v2, "\n\n")
```

ii)

```
# Matrix Operations
m <- matrix(1:9, nrow = 3, byrow = TRUE)
v <- c(1, 2, 3)
cat("Matrix:\n")
print(m)
cat("\nVector:", v, "\n\n")
cat("Matrix * Vector:\n")
print(m * v)
cat("\nMatrix / Vector:\n")
print(m / v)
```

Expected Output:**i)**

```
Vector 1: 10 20 30
Vector 2: 2 4 5
Addition: 12 24 35
Subtraction: 8 16 25
Multiplication: 20 80 150
Division: 5 5 6
```

ii)

Matrix:

```
      [,1] [,2] [,3]
[1,]  1    2    3
[2,]  4    5    6
[3,]  7    8    9
```

Matrix * Vector:

```
      [,1] [,2] [,3]
[1,]  1    2    3
[2,]  8   10   12
[3,] 21   24   27
```

Matrix / Vector:

```
      [,1]      [,2]      [,3]
[1,] 1.000000 2.000000 3
[2,] 2.000000 2.500000 3
[3,] 2.333333 2.666667 3
```

6. Write R command to**i) Illustrates the usage of Vector subsetting and Matrix subsetting****ii) Write a program to create an array of 3 X 3 matrixes with 3 rows and 3 columns.****Program:****i)**

```
# -----  
# Vector Subsetting  
# -----  
v <- c(10, 20, 30, 40, 50)  
cat("Original Vector:", v, "\n")  
# Access single element  
cat("2nd Element:", v[2], "\n")  
# Access range  
cat("Elements from 2 to 4:", v[2:4], "\n")  
# Exclude element  
cat("Exclude 3rd Element:", v[-3], "\n\n")  
  
# -----  
# Matrix Subsetting  
# -----  
m <- matrix(1:9, nrow = 3, byrow = TRUE)  
cat("Matrix:\n")  
print(m)  
# Access element  
cat("\nElement at (2,3):", m[2,3], "\n")  
# Access row  
cat("First Row:", m[1, ], "\n")  
# Access column  
cat("Second Column:", m[,2], "\n")
```

ii)

```
# Create 3x3x3 array  
arr <- array(1:27, dim = c(3, 3, 3))  
cat("3D Array (3 matrices of 3x3):\n")  
print(arr)
```

Expected Output:**i)**

By Mr.BALAJI B K

```
Original Vector: 10 20 30 40 50
2nd Element: 20
Elements from 2 to 4: 20 30 40
Exclude 3rd Element: 10 20 40 50
Matrix:
[,1] [,2] [,3]
[1,] 1 2 3
[2,] 4 5 6
[3,] 7 8 9
Element at (2,3): 6
First Row: 1 2 3
Second Column: 2 5 8
```

ii)

```
, 1
[,1] [,2] [,3]
[1,] 1 4 7
[2,] 2 5 8
[3,] 3 6 9
, , 2[,1] [,2] [,3]
[1,] 10 13 16
[2,] 11 14 17
[3,] 12 15 18
, , 3
[,1] [,2] [,3]
[1,] 19 22 25
[2,] 20 23 26
[3,] 21 24 27
```

7. Write an R program to draw

i) Pie chart ii) 3D Pie Chart iii) Bar Chart along with chart legend by considering suitable CSV file.

Program:

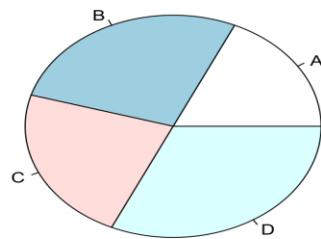
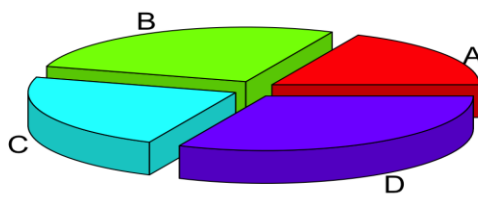
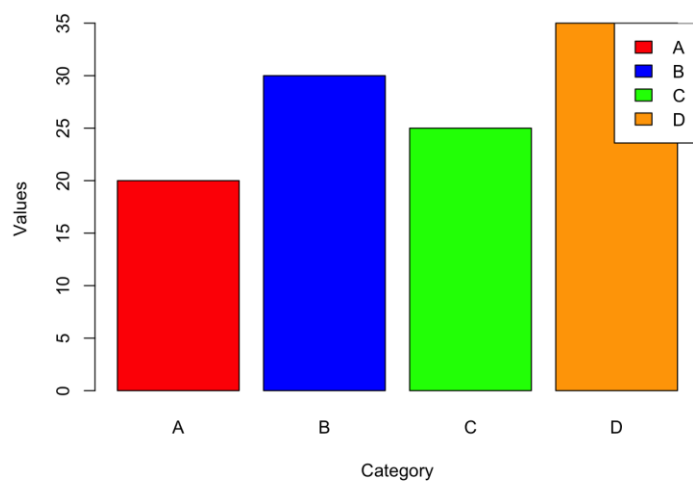
```
# Load required package
if(!require(plotrix)){
install.packages("plotrix")
library(plotrix)
}

# Create sample data
data <- data.frame(
Category = c("A", "B", "C", "D"),
Values = c(20, 30, 25, 35)
)

# Pie Chart
pie(data$Values,
labels = data$Category,
main = "Pie Chart")

# 3D Pie Chart
pie3D(data$Values,
labels = data$Category,
main = "3D Pie Chart",
explode = 0.1)

# Bar Chart with Legend
colors <- c("red", "blue", "green", "orange")
barplot(data$Values,
names.arg = data$Category,col = colors,
main = "Bar Chart",
xlab = "Category",
ylab = "Values")
legend("topright",
legend = data$Category,
fill = colors)
```

Expected Output:**Pie Chart****3D Pie Chart****Bar Chart**

8.Create a CSV file having Speed and Distance attributes with 1000 records. Write R program to draw

i) Box plots ii) Histogram iii) Line Graph iv) Multiple line graphs v) Scatter plot

Program:

```
# Step 1: Generate Data
set.seed(123)
speed <- runif(1000, 20, 100)
distance <- speed * 2 + rnorm(1000)
df <- data.frame(speed, distance)

# Step 2: Save CSV File
write.csv(df, "speed_distance.csv", row.names = FALSE)

# Step 3: Read CSV File
data <- read.csv("speed_distance.csv")

# Step 4: Box Plot
boxplot(data$speed,
  main = "Box Plot of Speed",
  ylab = "Speed",
  col = "lightblue")

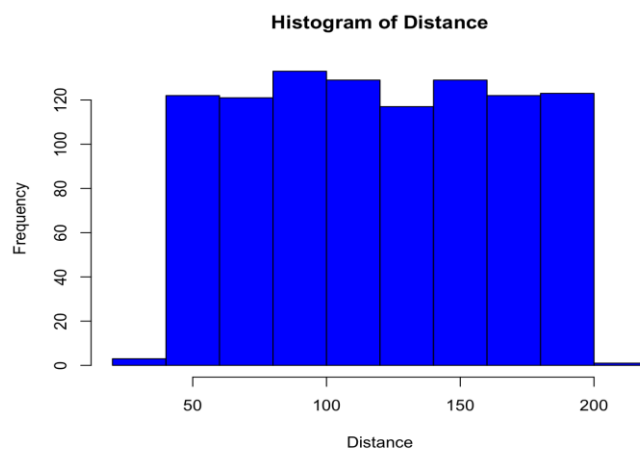
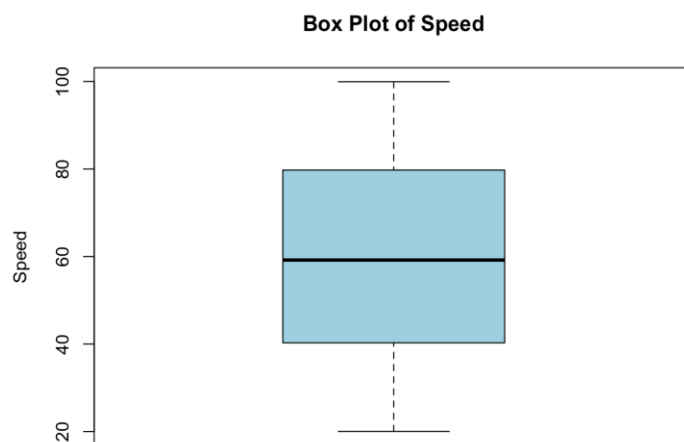
# Step 5: Histogram
hist(data$distance,
  main = "Histogram of Distance",
  xlab = "Distance",
  col = "blue")

# Step 6: Line Graph
plot(data$speed,
  type = "l",
  main = "Line Graph of Speed",
  xlab = "Index",
  ylab = "Speed",
  col = "green")

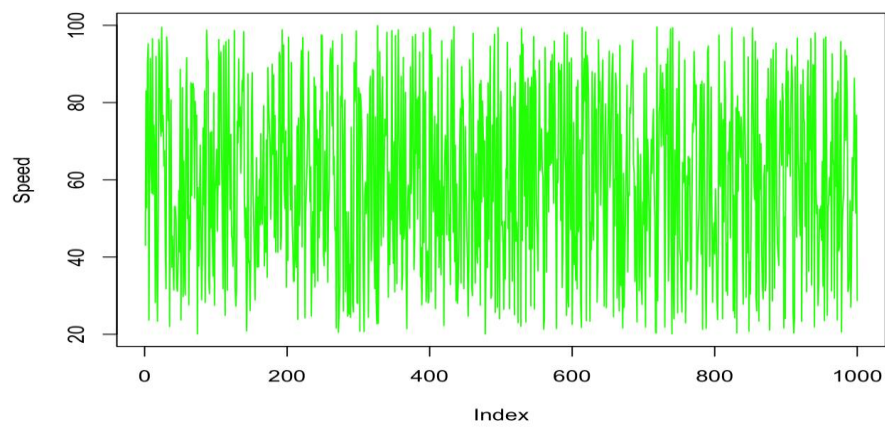
# Step 7: Multiple Line Graph
plot(data$speed,
  type = "l",
  col = "blue",
  main = "Multiple Line Graph",
  xlab = "Index",
  ylab = "Values")
lines(data$distance, col = "red")
legend("topright",
```

```
legend = c("Speed", "Distance"),  
col = c("blue", "red"),  
lty = 1)  
  
# Step 8: Scatter Plot  
plot(data$speed, data$distance,  
main = "Scatter Plot",  
xlab = "Speed",  
ylab = "Distance",  
col = "darkgreen")
```

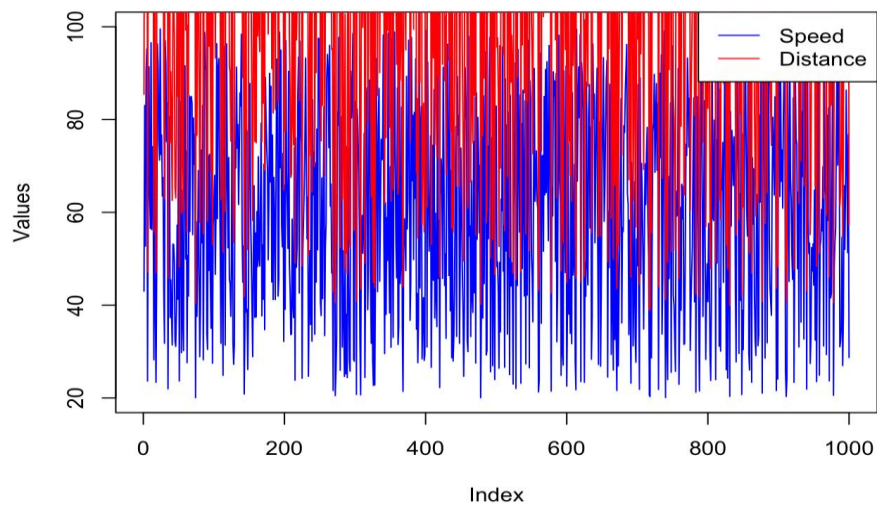
Expected Output:



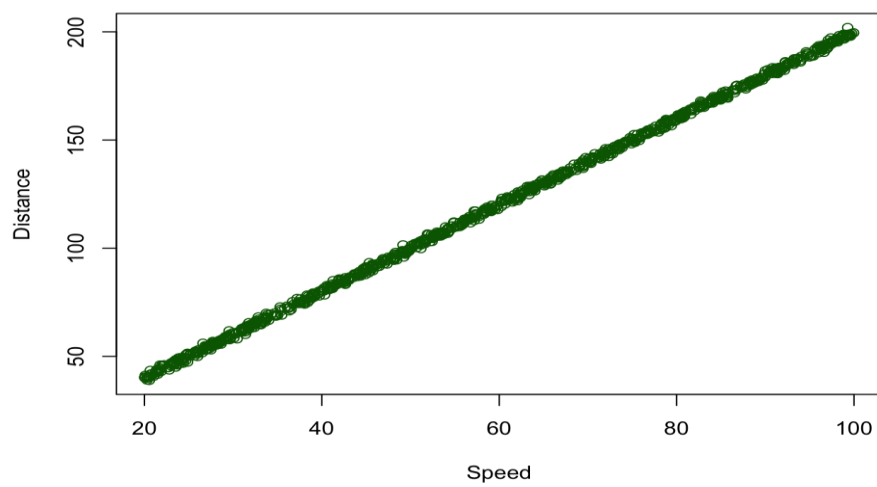
Line Graph of Speed



Multiple Line Graph



Scatter Plot



9.To demonstrate the relation between the cars speed and the distance.

Program:

```
# Load dataset
data(cars)

# Scatter plot
plot(cars$speed, cars$dist,
     main = "Speed vs Distance",
     xlab = "Speed",
     ylab = "Distance",
     col = "blue",
     pch = 19)

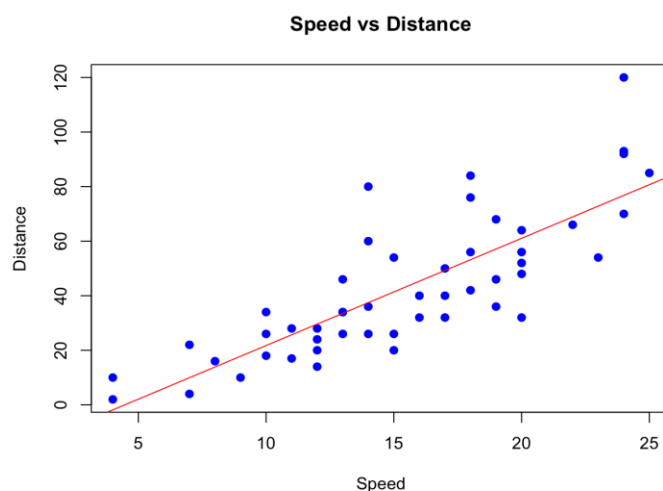
# Linear regression
model <- lm(dist ~ speed, data = cars)

# Add regression line
abline(model, col = "red")

# Correlation
correlation <- cor(cars$speed, cars$dist)
cat("Correlation:", correlation)
```

Expected Output:

Correlation: 0.8068949



10.Implement different data structures in R (Vectors, Lists, Data Frames).**Program:**

```
# -----  
# VECTOR OPERATIONS  
# -----  
  
v <- c(10, 20, 30, 40, 50)  
  
cat("Vector:\n")  
print(v)  
  
cat("Length:", length(v), "\n")  
cat("Sum:", sum(v), "\n")  
cat("Mean:", mean(v), "\n")  
  
# Subsetting  
cat("Elements > 25:\n")  
print(v[v > 25])  
  
# -----  
# LIST OPERATIONS  
# -----  
  
l <- list(  
  name = "Data Structures",  
  marks = c(80, 90, 85),  
  status = TRUE  
)  
  
cat("\nList:\n")  
print(l)  
  
# Access elements  
cat("Name:", l$name, "\n")  
  
# Modify list  
l$grade <- "A"
```

```
cat("Updated List:\n")
print(l)

# -----
# MATRIX OPERATIONS
# -----

m <- matrix(1:9, nrow = 3, byrow = TRUE)

cat("\nMatrix:\n")
print(m)

# Matrix operations
cat("Transpose:\n")
print(t(m))

cat("Row sums:\n")
print(rowSums(m))

cat("Column sums:\n")
print(colSums(m))

# -----
# ARRAY OPERATIONS
# -----

arr <- array(1:24, dim = c(3, 4, 2))

cat("\nArray:\n")
print(arr)

cat("Dimensions of array:", dim(arr), "\n")

# -----
# DATA FRAME OPERATIONS
# -----

df <- data.frame(
  Name = c("A", "B", "C", "D"),
  Marks = c(85, 90, 88, 92)
)
```

```
cat("\nData Frame:\n")
print(df)

# Add new column
df$Grade <- c("A", "A+", "A", "A+")

# Filter data
cat("\nMarks greater than 85:\n")
print(df[df$Marks > 85, ])

# Structure and summary
cat("\nStructure:\n")
str(df)

cat("\nSummary:\n")
print(summary(df))
```

Expected Output:

```
Vector:
[1] 10 20 30 40 50
Length: 5
Sum: 150
Mean: 30
Elements > 25:
[1] 30 40 50

List:
$name
[1] "Data Structures"

$marks
[1] 80 90 85

$status
[1] TRUE

Name: Data Structures
Updated List:
$name
[1] "Data Structures"

$marks
[1] 80 90 85
```

```
$status  
[1] TRUE
```

```
$grade  
[1] "A"
```

Matrix:

```
      [,1] [,2] [,3]  
[1,]    1    2    3  
[2,]    4    5    6  
[3,]    7    8    9
```

Transpose:

```
      [,1] [,2] [,3]  
[1,]    1    4    7  
[2,]    2    5    8  
[3,]    3    6    9
```

Row sums:

```
[1]  6 15 24
```

Column sums:

```
[1] 12 15 18
```

Array:

```
, , 1
```

```
      [,1] [,2] [,3] [,4]  
[1,]    1    4    7   10  
[2,]    2    5    8   11  
[3,]    3    6    9   12
```

```
, , 2
```

```
      [,1] [,2] [,3] [,4]  
[1,]   13   16   19   22  
[2,]   14   17   20   23  
[3,]   15   18   21   24
```

Dimensions of array: 3 4 2

Data Frame:

```
  Name Marks  
1    A    85  
2    B    90  
3    C    88
```

4 D 92

Marks greater than 85:

	Name	Marks	Grade
2	B	90	A+
3	C	88	A
4	D	92	A+

Structure:

```
'data.frame': 4 obs. of 3 variables:  
 $ Name : chr "A" "B" "C" "D"  
 $ Marks: num 85 90 88 92  
 $ Grade: chr "A" "A+" "A" "A+"
```

Summary:

Name	Marks	Grade
Length:4	Min. :85.00	Length:4
Class :character	1st Qu.:87.25	Class :character
Mode :character	Median :89.00	Mode :character
	Mean :88.75	
	3rd Qu.:90.50	
	Max. :92.00	

11. Write an R program to read a csv file and analyze the data in the file using EDA (Explorative Data Analysis) techniques.**Program:**

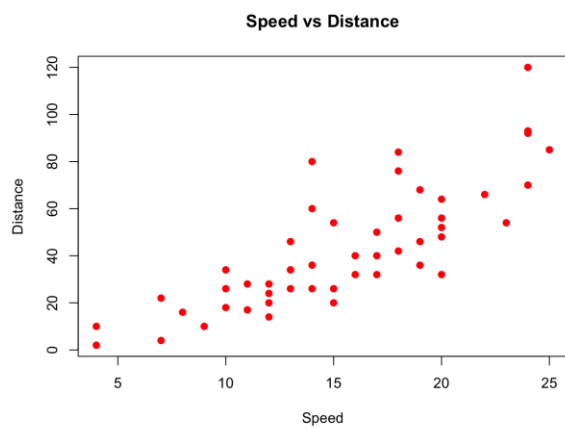
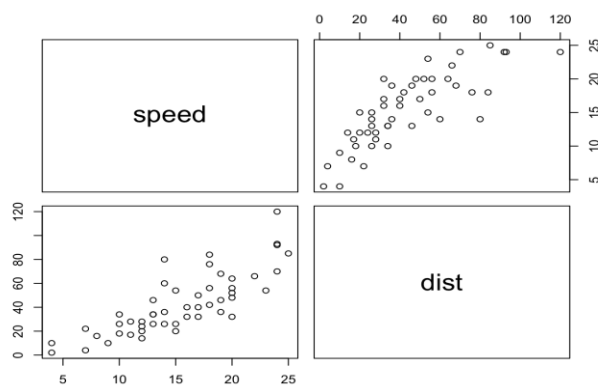
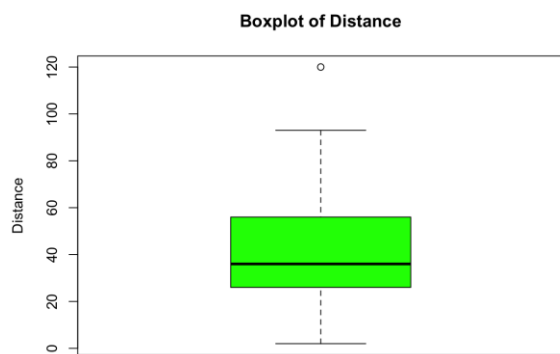
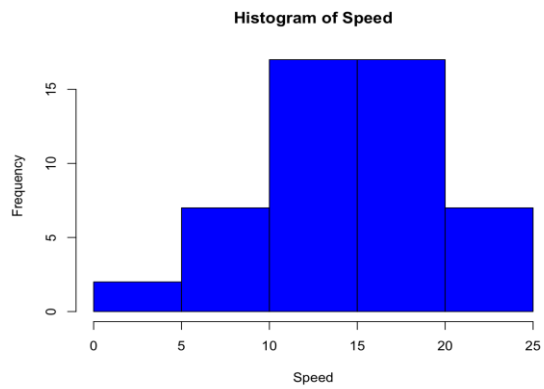
```
# -----  
# Load Built-in Dataset  
# -----  
  
data(cars)  
  
# -----  
# View Data  
# -----  
  
head(cars)  
tail(cars)  
  
# -----  
# Structure and Summary  
# -----  
  
str(cars)  
summary(cars)  
  
# -----  
# Check Missing Values  
# -----  
  
colSums(is.na(cars))  
# -----  
# Statistical Measures  
# -----  
  
colMeans(cars)  
apply(cars, 2, sd)  
# -----  
# Correlation  
# -----  
  
cor(cars)
```



```
# -----  
# Visualization  
# -----  
  
# Histogram  
hist(cars$speed,  
      main = "Histogram of Speed",  
      xlab = "Speed",  
      col = "blue")  
  
# Boxplot  
boxplot(cars$dist,  
         main = "Boxplot of Distance",  
         ylab = "Distance",  
         col = "green")  
  
# Scatter Plot  
plot(cars$speed, cars$dist,  
      main = "Speed vs Distance",  
      xlab = "Speed",  
      ylab = "Distance",  
      col = "red",  
      pch = 19)  
  
# Pair Plot (Advanced EDA)  
pairs(cars)
```

Expected Output:

```
data.frame':   50 obs. of  2 variables:  
 $ speed: num  4 4 7 7 8 9 10 10 10 11 ...  
 $ dist : num  2 10 4 22 16 10 18 26 34 17 ...
```



12. Write an R program to illustrate Linear Regression and Multi linear Regression considering suitable CSV file.**Program:**

```
# -----  
# Linear Regression using cars dataset  
# -----  
data(cars)  
  
# Create Linear Regression Model  
linear_model <- lm(dist ~ speed, data = cars)  
  
cat("Linear Regression Model:\n")  
print(summary(linear_model))  
  
# Plot with Regression Line  
plot(cars$speed, cars$dist,  
      main = "Linear Regression (Speed vs Distance)",  
      xlab = "Speed",  
      ylab = "Distance",  
      col = "blue",  
      pch = 19)  
  
abline(linear_model, col = "red", lwd = 2)  
  
# -----  
# Multiple Linear Regression  
# -----  
  
# Create sample dataset  
data2 <- data.frame(  
  x1 = c(1, 2, 3, 4, 5),  
  x2 = c(2, 1, 3, 5, 4),  
  y = c(5, 7, 9, 13, 12)  
)  
  
# Create Multiple Regression Model  
multi_model <- lm(y ~ x1 + x2, data = data2)  
  
cat("\nMultiple Linear Regression Model:\n")  
print(summary(multi_model))
```

Expected Output:

Linear Regression Model:

```
Call: lm(formula = dist ~ speed, data = cars)
```

```
Residuals: Min 1Q Median 3Q Max -29.069 -9.525 -2.272 9.215
43.201
```

```
Coefficients: Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept) -17.5791 6.7584 -2.601 0.0123 *
```

```
speed 3.9324 0.4155 9.464 1.49e-12 ***
```

```
Signif. codes: 0 '0.001' '0.01' '0.05' '.' 0.1 ' ' 1
```

```
Residual standard error: 15.38 on 48 degrees of freedom
```

```
Multiple R-squared: 0.6511, Adjusted R-squared: 0.6438 F-
statistic: 89.57 on 1 and 48 DF, p-value: 1.49e-12
```

Multiple Linear Regression Model:

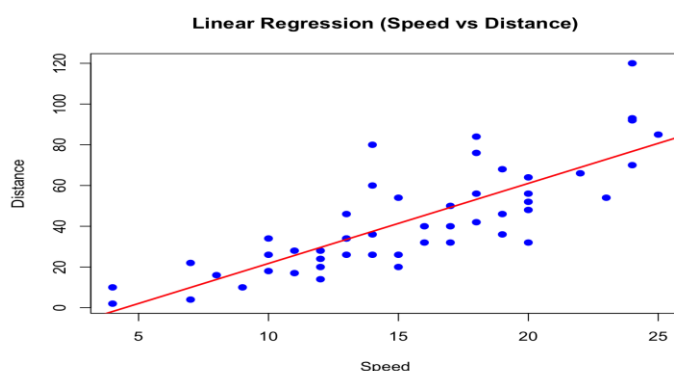
```
Call: lm(formula = y ~ x1 + x2, data = data2)
```

```
Residuals: 1 2 3 4 5 -0.7 0.8 -0.2 0.8 -0.7
```

```
Coefficients: Estimate Std. Error t value Pr(>|t|) (Intercept)
2.7000 1.1747 2.298 0.148 x1 1.3333 0.5652 2.359 0.142 x2
0.8333 0.5652 1.474 0.278
```

```
Residual standard error: 1.072 on 2 degrees of freedom
```

```
Multiple R-squared: 0.9487, Adjusted R-squared: 0.8973 F-
statistic: 18.48 on 2 and 2 DF, p-value: 0.05134
```



13.Create a data set and do statistical analysis on the data using R.

Program:

```
# Create Dataset
data <- c(10, 20, 30, 40, 50, 60)

# Display Data
cat("Dataset:\n")
print(data)

# Mean
cat("Mean:", mean(data), "\n")

# Median
cat("Median:", median(data), "\n")

# Variance
cat("Variance:", var(data), "\n")

# Standard Deviation
cat("Standard Deviation:", sd(data), "\n")

# Minimum and Maximum
cat("Minimum:", min(data), "\n")
cat("Maximum:", max(data), "\n")

# Summary
cat("Summary:\n")
print(summary(data))

# Histogram
hist(data,
      main = "Histogram",
      xlab = "Values",
      col = "blue")

# Boxplot
boxplot(data,
        main = "Boxplot",
```

```
col = "green")
```

Expected Output:

Dataset:

```
[1] 10 20 30 40 50 60
```

Mean: 35

Median: 35

Variance: 350

Standard Deviation: 18.70829

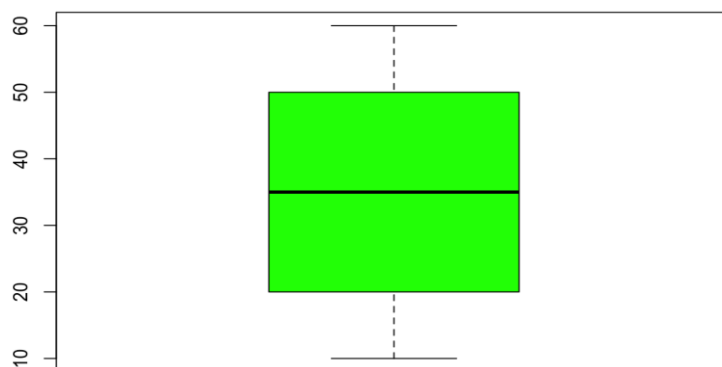
Minimum: 10

Maximum: 60

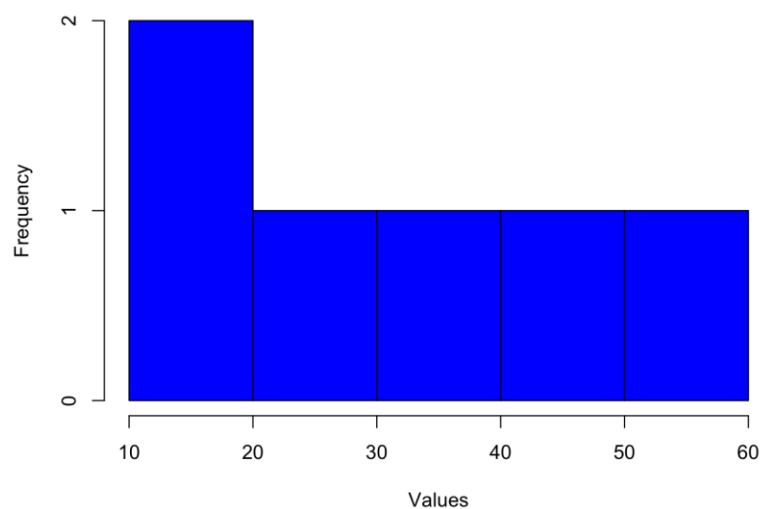
Summary:

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
10.0	22.5	35.0	35.0	47.5	60.0

Boxplot



Histogram



14. Write R program to find Correlation and Covariance

Program:

```
# Create Data
x <- c(10, 20, 30, 40, 50)
y <- c(15, 25, 35, 45, 55)

# Display Data
print(x)
print(y)

# Covariance
covariance <- cov(x, y)
cat("Covariance:", covariance, "\n")

# Correlation
correlation <- cor(x, y)
cat("Correlation:", correlation, "\n")
```

Expected Output:

```
[1] 10 20 30 40 50
[1] 15 25 35 45 55
Covariance: 250
Correlation: 1
```

15. Write R program to build classification model using KNN algorithm.**Program:**

```
# Load required package
library(class)
# Load dataset
data(iris)
# Split data
set.seed(123)
index <- sample(1:nrow(iris), 100)
train_data <- iris[index, ]
test_data <- iris[-index, ]
# Prepare data
train_x <- train_data[, 1:4]
test_x <- test_data[, 1:4]
train_y <- train_data$Species

# Apply KNN
pred <- knn(train = train_x,
  test = test_x,
  cl = train_y,
  k = 3)
# -----
# Show Predicted and Actual Values
# -----
cat("Predicted Values (First 10):\n")
print(head(pred, 10))
cat("\nActual Values (First 10):\n")
print(head(test_data$Species, 10))
# -----
# Confusion Matrix
# -----
cat("\nConfusion Matrix:\n")
cm <- table(Predicted = pred, Actual = test_data$Species)
print(cm)
# -----
# Accuracy
# -----
accuracy <- sum(pred == test_data$Species) / nrow(test_data)
cat("\nAccuracy:", accuracy)
```


Expected Output:

Predicted Values (First 10):

```
[1] setosa setosa setosa setosa setosa setosa setosa setosa setosa  
setosa setosa
```

Levels: setosa versicolor virginica

Actual Values (First 10):

```
[1] setosa setosa setosa setosa setosa setosa setosa setosa setosa  
setosa setosa
```

Levels: setosa versicolor virginica

Confusion Matrix:

	Actual		
Predicted	setosa	versicolor	virginica
setosa	16	0	0
versicolor	0	19	0
virginica	0	2	13

Accuracy: 0.96,

16. Write R program to build clustering model using K-mean algorithm.**Program:**

```
# Load dataset
data(iris)

# Use only numeric columns
data1 <- iris[, 1:4]

# Apply K-Means Clustering (k = 3)
set.seed(123)
kmeans_model <- kmeans(data1, centers = 3)

# View cluster details
print(kmeans_model)

# Cluster assignments
clusters <- kmeans_model$cluster

# Plot clusters
plot(data1[,1], data1[,2],
     col = clusters,
     main = "K-Means Clustering",
     xlab = "Sepal Length",
     ylab = "Sepal Width",
     pch = 19)
```

Expected Output:

K-means clustering with 3 clusters of sizes 50, 62, 38

Cluster means:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	5.006000	3.428000	1.462000	0.246000
2	5.901613	2.748387	4.393548	1.433871
3	6.850000	3.073684	5.742105	2.071053

Clustering vector:

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
[49] 1 1 2 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
[97] 2 2 2 2 3 2 3 3 3 3 2 3 3 3 3 3 3 2 2 3 3 3 3 2
3 2 2 3 3 3 3 2 3 3 3 3 2 3 3 3 2 3 3 3 3 3 3 3
[145] 3 3 2 3 3 2
```

Within cluster sum of squares by cluster:

```
[1] 15.15100 39.82097 23.87947
(between_SS / total_SS = 88.4 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"
"tot.withinss" "betweenss"
[7] "size"         "iter"         "ifault"
```